Milestone 3 — Chunked Prefill + Radix Prefix Cache + Retraction

Author: Hiva Mohammadzadeh **Model:** `Qwen/Qwen3-8B` **GPU:** NVIDIA L4 (24 GB) on AWS `g6.4xlarge` **Software:** Deep Learning OSS Nvidia Driver AMI GPU PyTorch 2.7 (Ubuntu 22.04), `flash-attn 2.8.3`, torch 2.7.0+cu128, CUDA 12.8.

> **Note on `--page-size`.** The milestone command line uses `--page-size 32`. `flash_attn 2.x`'s paged-attention kernel (`flash_attn_varlen_func(..., block_table=...)`) requires `page_size % 256 == 0` on Ada — with `--page-size 32` the new M3 prefill path raises *"Paged KV cache block size must be divisible by 256"*. All M3 runs in this report use `--page-size 256`, the smallest valid page size for flash_attn 2.8 on L4 (the same constraint M2 hit). Switching to flashinfer would lift the constraint but is out of scope.

Design and implementation

Milestone 3 layers **three additions** on top of the M2 paged engine, plus one structural change (lazy KV allocation) that makes the other three meaningful.

Lazy KV allocation (structural prerequisite)

M2 reserved `prompt_len + max_new_tokens` pages per request at admission. That worst-case reservation crushes concurrency — most requests stop well before `max_new_tokens`, but the pages stay pinned. M3 switches to **lazy allocation in paged mode unconditionally**:

- `_setup_paged_request(req)` allocates only **prompt** pages.
- `paged_decode_step` calls `_ensure_decode_page(req)` before each forward, which appends one more page from the pool when `cache_len % page_size == 0` and `cache_len > 0`.

The cost: `pool.allocate(1)` can raise `KVOutOfMemory` mid-decode. The benefit: it makes the radix cache effective (pages aren't burned on worst-case decode reservations) and creates the OOM the retraction bonus has to handle.

Part A — Chunked prefill (`--prefill-chunk-size N`)

Single-request, one chunk per scheduler step. The scheduler keeps a `self._prefilling: Request | None` slot; while occupied, no new admissions. Each chunk is one `flash_attn_varlen_func` call with:

```
cu_seqlens_q = [0, chunk_len]            # new tokens this step
cu_seqlens_k = [0, prefill_offset + chunk_len]   # full so-far
block_table  = request's full page_table  # cached prefix + new
```

The new chunk's K/V is scattered into the pool *before* the kernel runs (via `slot_mapping`); the kernel reads K/V for the full sequence back from the pool via `block_table`. Already-running requests continue decoding in parallel during the chunked prefill — per-step Q-token cost is bounded by `chunk_size`, decode adds at most `running_count` more single-token forwards.

Part B — Radix prefix cache (`--disable-radix-cache` to turn off)

`miniengine/radix_cache.py` implements the tree:

```
class RadixNode:
    parent:   RadixNode | None
    children: dict[tuple[int, ...], RadixNode]   # keyed by first-page tokens
    key:      list[int]                           # tokens on edge (page-aligned)
    pages:    list[int]                           # KV pages, one per page_size tokens
    ref_count: int                                # locked descendants
    last_access: float                            # LRU
```

Children are keyed by `tuple(key[:page_size])` (not just first token). This is non-obvious but necessary: two prompts can share a single chat-template token but diverge inside the first page, which crashes a first-token-keyed tree (caught the hard way on the first L4 bench).

- **`match_prefix(tokens)`** walks page-by-page; matches in page-aligned chunks per edge; stops at the first page-aligned divergence.
- **`insert_and_return(tokens, pages)`** walks down, splits an edge in two when the input diverges mid-edge, attaches new branches. Pages for tokens that already exist on the same path become **redundant** and are returned to the caller; the caller frees them back to the pool. No mid-flight `page_table` swap.
- **`evict(n)`** — min-heap over unlocked leaves; oldest first; freed leaves' parents may become leaves themselves and get re-pushed.
- **`inc/dec_lock_ref(node)`** walks parent chain; locks pin the matched subtree and every ancestor against eviction while a request borrows.

The engine wires the cache into both prefill and finish:

```
# at admission (before any forward pass)
match = self.radix_cache.match_prefix(req.input_ids)
self.radix_cache.inc_lock_ref(match.last_node)
req.page_table = match.matched_pages + self.kv_pool.allocate(...)
req.cache_len  = match.matched_tokens
req.cache_hit_tokens = match.matched_tokens  # served via /usage

# after prefill completes
self.radix_cache.insert_and_return(req.input_ids[:page_aligned], req.page_table[:n])

# on finish (free_paged_request)
self.radix_cache.dec_lock_ref(req.matched_node)
self.radix_cache.insert_and_return(prompt+output[:page_aligned], all_pages[:n])
self.kv_pool.free(redundant + tail)
```

`KVMemoryPool.allocate(n)` calls `cache.evict(n - free)` before raising `KVOutOfMemory`. The pool's `_free` deque never holds cache-borrowed pages; eviction is the only path back.

Bonus — Retraction (`--enable-retraction`)

```
def _step_paged_decode(self, finished):
    while True:
        try:
            token_ids = self.engine.paged_decode_step(self.running)
            break
        except KVOutOfMemory:
            if not self._retract_one_victim():
                logger.error("KV OOM, no eligible victim")
                return
            # retry decode with one fewer running request
```

Victim policy: **youngest by `arrival_time`**, tie-break by largest remaining work (`max_new_tokens - num_output_tokens`). The chunked- prefill request lives in `self._prefilling`, never in `self.running`, so it's never a candidate. The victim's matched-node lock is decremented before its pages return to the pool; the cached prefix itself stays available for the next admission.

Performance

Setup. All benchmarks below run against `Qwen/Qwen3-8B` on a single L4, `--mode paged --mem-fraction-static 0.85 --page-size 256`. KV pool holds 98 × 256-token pages (3.73 GB). bench_cache and bench_serving clients run on the same host. Source files in `bench-out/`.

Part B headline — shared workload (Deliverable #4)

10 groups × 10 questions, `--shared-prefix-len 2000 --concurrency 4 --max-tokens 64`. Cache-on vs `--disable-radix-cache`.

Metric	Cache off	Cache on	Speedup
Wall time	103.22 s	40.45 s	**2.55x**
Throughput	0.97 req/s	2.47 req/s	**2.55x**
TTFT p50	3276 ms	226 ms	**14.5x**
TTFT p99	3637 ms	1790 ms	2.03x
Gen tok/s	12	37	3.08x
Cache hit rate	0.0%	**86.8%**	—

Target met: >= 2x throughput AND >= 2x TTFT. Both exceeded by wide margins. Per-group breakdown: groups 0,2,4-9 hit ~88.7%; groups 1 and 3 land at 78.9% — those were unlucky enough to be in the first concurrent batch-of-4 that all cold-missed before anyone's prefix had been inserted. Once the cache warms, every subsequent group sees ~8 of 9 prefix pages reused.

Part B prefix-length sweep (Deliverable #5)

Same workload, cache on, sweeping `--shared-prefix-len`:

Prefix length	Wall time	Throughput	TTFT p50	TTFT p99	Hit rate
200	40.68 s	2.46 req/s	296 ms	455 ms	**0.0%**
500	36.00 s	2.78 req/s	171 ms	301 ms	83.7%
2000	40.60 s	2.46 req/s	156 ms	243 ms	**98.6%**
4000	135.47 s	0.74 req/s	248 ms	1989 ms	91.7%

Three regimes the sweep reveals. (a) **L=200** is *shorter than one page* (page_size = 256); the cache structurally can't insert anything page-aligned, hit rate is 0% — this is the page-granularity floor. (b) **L=500..2000** is the sweet spot: hit rate climbs to 98.6%, TTFT p50 drops from 296 ms to 156 ms — almost 2x. (c) **L=4000** keeps the high hit rate (91.7%) but throughput *drops* to 0.74 req/s because the per-step decode cost grows linearly with the context length the kernel has to read every step (18 cached pages per request). TTFT stays low — what we wanted — but per-request *latency* increases because each request now has more decode work.

Part B multi-turn (Deliverable #6)

16 sessions × 10 turns, `--concurrency 8`. Per the spec (*"pick `--turns-per-session` and per-turn generation length so the cumulative cached prefix dominates"*) we ran two configurations: the default `--max-tokens 64` and a prefill-dominant `--max-tokens 16`.

Headline (`--max-tokens 16`):

Metric	Cache off	Cache on	Speedup
Wall time	39.24 s	34.57 s	1.14x
Throughput	4.08 req/s	4.63 req/s	**+13.5%**
TTFT p50	642 ms	365 ms	**-43%**
TTFT p99	1048 ms	861 ms	-18%
Latency p50	1953 ms	1721 ms	-12%
Hit rate	0%	**44.0%**	—

Default config (`--max-tokens 64`) for reference:

Metric	Cache off	Cache on	Speedup
Throughput	1.60 req/s	1.95 req/s	+22%
TTFT p50	274 ms	179 ms	-35%
Hit rate	0%	49.0%	—

Per-turn breakdown at `--max-tokens 16` — hit rate climbs then *falls*:

Turn	Prompt tok	Hit tok	Hit rate	TTFT p50
0	2275	0	0.0%	355 ms
1	3011	0	0.0%	175 ms
2	3884	0	0.0%	178 ms
3	4726	1024	21.7%	212 ms
4	5496	3072	55.9%	204 ms
5	6389	4096	**83.3%**	166 ms
6	7390	4096	75.1%	338 ms
7	8317	4608	55.4%	375 ms
8	9250	6144	63.0%	348 ms
9	10155	6912	58.4%	376 ms

Hit rate peaks at 83.3% on turn 5, then *decreases* through turns 6-9. The cause is **KV-pool pressure**: 16 concurrent sessions × ~27 pages per session at turn 9 = ~430 pages, but the pool only holds 98 (`mem_fraction_static 0.85` on a 24 GB L4 after the 8 B-parameter weights). LRU eviction starts kicking in once cumulative cached state exceeds the pool, so sessions lose their own earlier prefixes between turns and have to re-prefill them on re-admission. This is a workload/pool sizing limit, not a cache-logic bug — at lower concurrency or longer pool budget the hit rate would keep climbing.

Target: spec asks for >= 50% throughput improvement and >= 50% TTFT reduction aggregated across turns. We hit:

- **TTFT p50 -43%** at `--max-tokens 16` (close to the -50% target), **-35%** at `--max-tokens 64`.
- **Throughput +13.5%** at `--max-tokens 16`, **+22%** at `--max-tokens 64`. *Short of the +50% target in both cases.*

The throughput target is the harder one for this workload. The cache halves the *prefill* compute on cached pages, but multi-turn requests at `max-tokens 16` already have a small prefill-to-decode ratio, and the prompts are short relative to `--shared-prefix-len 2000` in the shared workload (where we comfortably exceeded both targets). To hit +50% throughput on multi-turn would require either a much smaller `--max-tokens` (driving the decode share down further) or a much larger pool so the per-session cumulative prefix can stay cached through all turns. The TTFT side already demonstrates the cache is doing exactly what it should: short- circuiting the prefill of cached pages, which is the only thing the cache can do.

No-regression on default workload (Deliverable #7)

Not collected in this submission — server-side measurement only; plumbing identical to M2's bench_serving. Cache-off lookup path is a single `if self.radix_cache is None` branch, no overhead. Will fold in if time permits.

Part A — Chunked prefill (Deliverables #1, #2, #3)

Code and CLI flag (`--prefill-chunk-size`) implemented and syntactically verified against the existing paged kernel. End-to-end OOM and accuracy benchmarks not executed in time for this writeup.

Bonus — Retraction (Deliverables #8, #9)

Code and CLI flag (`--enable-retraction`) implemented; victim policy and edge cases (chunked-prefill never a victim; matched-node lock decremented on retraction) documented in §Design. Workload-side demonstration not executed in time.

Next Steps

Bottleneck 1 — Decode dominance at `--max-tokens 64`. The 50% throughput target wasn't reached at the default `--max-tokens 64` because decode dominates per-request time. Each turn does ~250 ms of decode and ~50–500 ms of prefill (growing with conversation length); the cache halves prefill, but decode is unchanged. Net throughput shift is ~22%.

Bottleneck 2 — Pool-size pressure at `--max-tokens 16`. Cutting `--max-tokens` to 16 makes prefill the larger share, *but* now the per-session cumulative cached state exceeds the pool. 16 concurrent sessions × ~27 pages of cached prefix per session at turn 9 = ~430 pages, against the pool's 98 pages. LRU eviction starts taking sessions' own prefixes back, hit rate peaks at 83% on turn 5 then *drops* to 58% by turn 9. With a larger pool (e.g. an L40S at 48 GB, or the same L4 reserved more aggressively) we expect the curve to keep climbing past 90% and throughput to follow.

Additional techniques implemented beyond required scope.

- **Lazy KV allocation.** Made unconditional in paged mode — M2's worst-case reservation is gone, freeing pool capacity for the cache to use. This is what makes the radix cache actually effective; with worst-case reservation, the pool fragments long before the cache can amortize.
- **Page-aligned dict keying.** Children are keyed by *first page tokens*, not first token. The textbook radix design uses first-character keys, but with page-aligned matching, two unrelated prompts sharing only the chat-template prefix collide on the first token while diverging within the first page. First-page keying makes them correct siblings. Caught during the first L4 bench when the engine crashed on `split.key[0]` for an empty key list — fixed and committed in `83ae1f2`.
- **Two-point cache insertion.** Prefix is inserted *both* after the prefill forward completes (so concurrent batchmates may still hit) *and* on request finish (so the assistant's response becomes available for the next multi-turn round). Pages that are duplicates of already-cached entries (`insert_and_return`'s `redundant_pages`) return to the pool — no mid-flight `page_table` swap, no lock juggling.
- **Safe-unwind on alloc failure.** `_setup_paged_request` increments the matched-node lock *before* it allocates. If `pool.allocate` raises `KVOutOfMemory`, the helper decrements the lock and zeroes `cache_hit_tokens` before re-raising — the scheduler can re-queue the request without leaking a pin.
- **Retraction with chunked-prefill carve-out.** The retraction loop draws victims from `self.running` only; the in-flight chunked- prefill request lives in `self._prefilling` and is never a victim. Its borrowed cache pages stay locked. If no eligible victim exists, the scheduler logs and drops the decode step instead of crashing — a soft failure mode under workloads larger than the pool can hold.